

Phase 1: Laying the Foundation

Before writing a single line of code, you need the right people and a clear understanding of who does what.

Step 1: Team Formation

Building an AI team is different from building a standard software team. You need a mix of data science, engineering, and domain expertise.

Key Roles to Fill:

- **Product Owner (PO):** This is a crucial role. The PO must understand both the business domain and the capabilities/limitations of AI. They are responsible for defining the "what" and prioritizing the backlog to deliver business value.
- **AI/ML Engineer(s):** These team members focus on the data science lifecycle: data exploration, feature engineering, model selection, training, and evaluation. They are concerned with accuracy, bias, and model performance.
- **ML/Ops Engineer(s) / Data Engineer(s):** This role is responsible for the infrastructure and pipelines. They build the systems to fetch and preprocess data, version-control models (e.g., using DVC), deploy models to production (inference), and monitor them for "drift" (when model performance degrades over time). This role is *critical* for moving from a prototype to a real product.
- **Software Engineer(s) / DevOps:** They integrate the AI model into the wider application, build APIs, handle the user interface, and manage the overall cloud infrastructure.
- **Domain Expert:** Whenever possible, have access to a subject matter expert (e.g., a doctor for a medical AI, a financial analyst for a fintech AI). They are invaluable for validating features, understanding data nuances, and defining what "good" looks like.

Team Size: Start with a small, cross-functional team of 5-7 people. A smaller team communicates more effectively and can move faster in the initial, high-uncertainty phase.

Step 2: Creating a RASCI Matrix

A RASCI matrix clarifies roles and responsibilities, preventing confusion and ensuring critical tasks don't fall through the cracks. RASCI stands for:

- **Responsible:** The person who does the work.
- **Accountable:** The person who signs off or is ultimately answerable (the "buck stops here"). Usually one person.
- **Supporting:** People who help the Responsible person get the work done.
- **Consulted:** People who need to give input *before* the work is done (two-way communication).
- **Informed:** People who need to be updated *after* a decision or action is taken (one-way communication).

How to build it:

1. **List Key Activities/Actions** (rows). For an AI project, examples are:
 - Define business problem & success metrics
 - Explore and acquire raw data
 - Clean and preprocess data
 - Engineer features
 - Design model architecture
 - Train and evaluate model
 - Create data/model versioning pipeline
 - Deploy model to production (API)
 - Integrate model into UI
 - Monitor model performance (drift, accuracy)
2. **List Project Roles** (columns). (e.g., Product Owner, AI Engineer, ML/Ops Eng., SW Eng., Domain Expert).
3. **Assign one R, one A, and any number of S, C, I per activity.**

Example Snippet of the Matrix:

Activity / Action	Product Owner	AI Engineer	ML/Ops Eng.	SW Eng.
Define business problem & metrics	A	C	I	I
Explore & acquire raw data	I	R	S	I
Clean & preprocess data	I	R	S	I
Train & evaluate model	I	R	S	I
Create data/model pipeline	I	C	R/A	I
Deploy model to production (API)	I	C	R	S
Monitor model performance	C	S	R/A	I

Step 3: Selection of Tools for Organizing Collaboration on AI

Choose tools that support transparency and the specific needs of AI work (data, models, experiments).

- **Project Management & Communication:**

- *Jira/Linear/Asana*: For managing epics, stories, and tasks (Backlog Planning).
- *Confluence/Notion/Miro*: For documentation, architecture diagrams, meeting notes, and the RASCI matrix itself.
- *Slack/Teams*: For daily, asynchronous communication.

- **Code & Model Collaboration:**

- *Git (GitHub/GitLab)*: For version control of code. Non-negotiable.
- **Crucial for AI: Data and Model Versioning.** Standard Git is not designed for large files.
 - *DVC (Data Version Control)*: A popular tool that works on top of Git to version datasets and models.
 - *Git LFS (Large File Storage)*: A Git extension for handling large files.
 - *Weights & Biases (WandB)/MLflow/Neptune.ai*: These are "MLOps" platforms specifically designed to track experiments. They log every training run, its parameters, code version, dataset used, and resulting metrics (accuracy, loss). This is essential for reproducibility and collaboration.

Phase 2: Planning the Work

This phase is about populating your new project management tool with a clear plan.

Step 4: Task Backlog Planning

This is the "traditional" part of the backlog, focusing on the software engineering and integration work.

- **Epics:** Large bodies of work.

- Example: "Data Pipeline Infrastructure," "Model Training Environment Setup," "User Interface for AI Feature," "Model Deployment & API."

- **User Stories:** Features from a user's perspective. Use the format: "As a [user], I want to [do something] so that [I get some value]."

- Example: "As a data scientist, I want to be able to trigger a training job from a single command so that I can iterate quickly."
- Example: "As an end-user, I want to see the AI's confidence score for its prediction so that I can gauge how much to trust it."
- Example: "As an ML Engineer, I want the model's prediction API to have a response time of <200ms so that the user experience is smooth."

Step 5: Data Backlog Planning

This is unique to AI. Data is the fuel, and its planning is just as important as feature planning.

- **Data Epics:**
 - "Data Acquisition & Labeling"
 - "Data Cleaning & Preprocessing"
 - "Feature Engineering & Selection"
- **Data Stories (more technical/exploratory):**
 - "Acquire initial dataset of customer transactions from Q1 2023." (Acceptance Criteria: Data is downloaded and accessible in the project's data lake).
 - "Profile the dataset to identify missing values, outliers, and class imbalance." (Acceptance Criteria: A data profiling report is created and shared with the team).
 - "Create a pipeline to clean text data by removing stopwords and performing stemming."
 - "Create a labeled dataset of 10,000 images for the object detection task." (This might involve a task for a domain expert or a labeling service).

Step 6: Outlining Ceremonies (The DDS Framework)

The Data Driven Scrum (DDS) framework adapts standard Scrum to the realities of AI. The key difference is that the work is not just about building features, but about generating *validated learnings*.

- **1. Sprint Planning (Time-boxed to ~2 hours for a 2-week sprint):**
 - **Goal:** Select a forecast of backlog items (both Task and Data stories) the team can complete.
 - **AI Twist:** The team commits to a **Sprint Goal**, which is often a hypothesis. E.g., *"Achieve 85% accuracy on the validation set for the fraud detection model"* or *"Successfully deploy the model to the staging environment and integrate it with the API."* The plan is built around achieving that goal.
- **2. Daily Sync (Time-boxed to 15 minutes):**
 - **Goal:** Inspect progress toward the Sprint Goal and adapt the plan.
 - **AI Twist:** Same three questions, but answers are highly technical.
 - *"What did I do yesterday?"* "I ran an experiment with a different neural network architecture. The training loss is decreasing, but I'm seeing signs of overfitting." / "I finished the data cleaning script for the new data source."
 - *"What will I do today?"* "I'm going to add a regularization layer to the model and re-run the training." / "I'll start working on the DVC pipeline to version this new cleaned dataset."
 - *"Do I have any blockers?"* "The training job on the GPU cluster keeps getting preempted. We need to look into spot instance policies." / "I'm waiting for the domain expert to label the new batch of data."
- **3. Sprint Review (Time-boxed to ~1 hour for a 2-week sprint):**
 - **Goal:** Demonstrate what was built and get feedback from stakeholders.
 - **AI Twist:** You are often demonstrating *results*, not just features. Show a demo of the model in action, but also present the key performance metrics. This is a conversation, not just a presentation.
 - *"We aimed to reach 85% accuracy. We currently have a model at 82%. We learned that data source X is very noisy and hurts performance. We recommend spending the next sprint cleaning that data."*
 - Show a before/after comparison of the model's performance. If it's a computer vision model, show examples of correct and incorrect predictions to build trust and understanding with stakeholders.
- **4. Sprint Retrospective (Time-boxed to 1 hour):**
 - **Goal:** Inspect the team's process and create a plan for improvements in the next sprint.
 - **Crucial AI Twist:** This is where you discuss the inherent **uncertainty and variability** of AI work. The standard "Did we deliver what we said we would?" question is often not the right one.
 - **Discuss Model Performance Variability:** "Our model's accuracy fluctuated a lot this sprint based on the random seed we used for training. How can we make our evaluation process more robust?" (e.g., by running multiple trials).
 - **Discuss Hypothesis Validation:** "We set out to improve accuracy with a new feature, but it only helped by 0.5%, which is within the noise. The learning is that this feature isn't valuable. Was the effort worth the learning?"
 - **Discuss Uncertainty:** "The data labeling took twice as long as expected because the labeling instructions were ambiguous. How can we improve our collaboration with the domain expert to get clearer guidelines upfront?"
 - **Celebrate Learnings, Not Just Successes:** A failed experiment that clearly disproves a hypothesis is a valuable outcome for a sprint. The retrospective is the place to acknowledge that this "failure" is actually progress.

Phase 3: Executing the First Sprints

Step 7: Planning the First Two Sprints

The goal of the first sprints is to build a foundation and prove the concept, not to deliver a polished feature.

• Sprint 1: The "Inception" Sprint

- **Goal:** Establish the development pipeline and get a first, very rough model running.
- **Tasks:**
 1. Set up the project repository (Git).
 2. Set up the development environment (e.g., Docker containers, cloud VM).
 3. Acquire a small, initial sample of data.
 4. Perform basic exploratory data analysis (EDA). Create a simple report.
 5. Set up the experiment tracking tool (e.g., WandB).
 6. **(The "Hello World" of AI):** Train an extremely simple baseline model (e.g., a very simple linear model or a pre-trained model used out-of-the-box) on the small dataset. The goal is not accuracy, but to have an *end-to-end pipeline* from data -> training script -> model evaluation.
- **Definition of Done:** A script exists that can take the sample data, train a model, and log a metric (even if it's a terrible one) to the experiment tracker.

• Sprint 2: The Data & Iteration Sprint

- **Goal:** Improve the data pipeline and iterate on the model to get a "proof of concept."
- **Tasks:**
 1. Based on Sprint 1's EDA, clean and preprocess the data more thoroughly.
 2. Acquire a larger, more representative dataset.
 3. Set up the data versioning pipeline (DVC).
 4. Engineer a few key features identified by the domain expert.
 5. Train a more sophisticated model (e.g., a random forest or a simple neural net).
 6. Systematically evaluate the model on a hold-out test set.
 7. *(Optional but good):* Create a very simple notebook or a basic Streamlit app to visualize the model's predictions.
- **Definition of Done:** A model exists that shows promising results on a test set, and the entire process (from fetching the correct data version to retraining) is reproducible.

Step 8: Review and Retro of the First Two Sprints

• After Sprint 1 (Review & Retro):

- **Review:** Show the team's setup. The demo is the pipeline itself. Explain the baseline model's performance (e.g., "Our baseline model gets 20% accuracy, which is just slightly better than random guessing. This gives us a floor to improve from."). Stakeholders might be underwhelmed; manage expectations by emphasizing that the *infrastructure* for rapid iteration is now in place.
- **Retro:** Focus on the setup process. Was it smooth? Were there permissions issues? Did everyone have the right tools? The action item might be to document the environment setup process.

• After Sprint 2 (Review & Retro):

- **Review:** This is more exciting. Present the improved model's metrics. Show the demo. Get feedback from stakeholders. "Is this prediction useful? What other features would you like to see?" This feedback directly feeds into the Product Backlog for Sprint 3.
- **Retro:** Focus on the data science process.
 - "Our experiments weren't well-organized. In Sprint 3, we'll use a more structured naming convention in our experiment tracker."
 - "The data cleaning took longer than expected. We underestimated the dirtiness of the 'location' field."
 - "We didn't have a chance to discuss the bias metrics we saw in the model for subgroup X. Let's put a story in the backlog to investigate this further." This last point is a perfect example of an AI-specific improvement.